

JAVA INTERVIEW PREPARATION

CHAPTER 31

JVM PROFILING AND OBSERVABILITY

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

JVM Profiling and Observability: JFR, JMC, jcmd, Thread Dumps, and Evidence-Driven Diagnosis

Senior / Lead Java Developer Reading Chapter

Production troubleshooting is an exercise in reducing uncertainty. Metrics show when behavior changed, profiles show where resources were consumed, dumps show current ownership and reachability, and traces connect work across services. No single tool explains every failure. Senior engineers choose the least intrusive evidence that can confirm or reject a specific hypothesis.

Observation Has Levels

```
metrics -> trend and alert
logs    -> discrete events and context
traces  -> request path across components
profiles-> sampled resource consumption over time
dumps   -> detailed point-in-time state
```

CPU at 95 percent is a symptom. A CPU profile identifies hot stacks. A thread dump can show whether the same stacks persist. JFR can correlate CPU, allocation, locks, I/O, GC, and thread events over a time window. Application metrics explain business impact.

Establish a Time Window and Baseline

Before attaching tools, record:

- Incident start time and deployment or configuration changes.
- Affected endpoints, tenants, nodes, and traffic volume.
- CPU, memory, GC, thread, queue, connection, and latency trends.
- Container throttling, restarts, and resource limits.
- A healthy comparison period or node.

Profiles without a relevant time window can precisely describe normal behavior while missing the incident.

jcmd as the Preferred Diagnostic Entry Point

`jcmd` sends supported diagnostic commands to a running HotSpot JVM.

```
jcmd -l
jcmd <pid> help
jcmd <pid> VM.version
jcmd <pid> VM.command_line
jcmd <pid> VM.flags
```

Always inspect `help` on the target JDK because commands, arguments, and impact vary by version and JVM implementation. Some commands require a safepoint or perform expensive heap traversal.

Useful categories include:

```
Thread.print
GC.heap_info
GC.class_histogram
GC.heap_dump
VM.classloader_stats
VM.native_memory
JFR.start, JFR.check, JFR.dump, JFR.stop
```

Do not run a high-impact command on every production node simultaneously. Confirm PID, node role, disk space, security, and expected pause.

Java Flight Recorder

JFR is a low-overhead event-recording framework built into the JDK. It can record CPU samples, allocation, GC, locks, thread parking, file and socket I/O, exceptions, class loading, and JVM events depending on configuration.

```
jcmd <pid> JFR.start name=incident settings=profile duration=120s \
    filename=/diagnostics/incident.jfr
```

The bundled `default.jfc` is intended for lower-overhead continuous recording; `profile.jfc` collects more detail with greater overhead and is normally used for shorter investigations. Exact overhead is workload- and configuration-dependent.

Continuous recording with a bounded disk repository can preserve evidence from before an alert. Starting a recording only after a brief incident has ended may be too late.

JDK Mission Control

JMC visualizes JFR recordings and provides automated analysis and pages for threads, memory, GC, locks, I/O, methods, and environment data.

Use automated findings as leads, not verdicts. A hot method may be correct work caused by excessive call volume. High allocation may be harmless if objects die cheaply. A lock event matters when it aligns with request latency and blocked owners.

```
symptom -> JMC view -> application question
high CPU -> method profiling -> expected algorithm or runaway repetition?
high allocation -> allocation stack -> temporary throughput or retained backlog?
lock time -> locks/threads -> who owns the lock and what is owner doing?
socket wait -> I/O -> slow dependency, timeout, or expected idle connection?
```

CPU Profiling

Sampling profiles periodically inspect thread stacks and estimate where CPU time is spent. They usually have lower distortion than instrumenting every method.

```
1000 samples
 420 JSON serialization
 260 regex matching
 180 business calculation
 140 other
```

Wall-clock or execution sampling and CPU sampling answer different questions. A method can dominate elapsed time while waiting for I/O but consume little CPU. Confirm the event type and profiler mode before interpreting percentages.

Flame graphs aggregate stack samples. Width represents sample frequency, not chronological duration. A wide frame may be called often from many paths. Read from callers toward callees and compare healthy and unhealthy recordings.

Allocation Profiling

Allocation profiles identify types and stack traces responsible for new objects.

```
request -> parse payload -> create temporary char[] and String
        -> map DTOs -> allocate collections
```

High allocation rate can increase GC frequency without causing a leak. Heap dumps show retained objects; allocation profiles show newly created objects. Use both when a structure is created rapidly and some instances are retained.

JFR allocation events can be sampled and may not account for every byte as an exact ledger. Use them to find dominant sites and confirm with metrics or targeted profiling.

Thread Dumps

A thread dump captures stacks and synchronization information at one instant.

```
jcmd <pid> Thread.print -l
```

Capture several dumps a few seconds apart. Persistent identical RUNNABLE stacks suggest a loop or hot computation. Many BLOCKED threads behind the same monitor point to a contended owner. WAITING workers may be healthy idle pool threads.

For Java 21 virtual-thread-heavy applications, use diagnostics designed to represent virtual threads, such as the target JDK's `Thread.dump_to_file` command where available. Standard ThreadMXBean monitoring is centered on platform threads and does not provide complete virtual-thread management.

Thread state is not diagnosis. Follow wait relationships and inspect the owner, executor, queue, or downstream operation.

GC Logs and JFR Together

GC logs provide exact collector phase and occupancy information, while JFR connects pauses to allocation, threads, CPU, and application behavior.

```
-Xlog:gc*:file=/diagnostics/gc.log:time,uptime,level,tags
```

Correlate by timestamps:

```
12:00:03 request latency rises
12:00:03.2 CPU throttling begins
12:00:04 G1 evacuation pause 180 ms
12:00:05 executor queue age rises
```

The GC pause may contribute without being the initiating cause. CPU throttling can lengthen concurrent GC and application work simultaneously.

Heap Histograms and Dumps

A histogram is useful for lower-cost class-count triage:

```
jcmd <pid> GC.class_histogram
```

A heap dump supports retained-size and GC-root analysis:

```
jcmd <pid> GC.heap_dump filename=/diagnostics/heap.hprof
```

Heap operations may be intrusive. Dumps need secure storage and enough disk. Histograms and dumps answer heap-retention questions, not direct native-memory growth.

Native Memory Tracking

If NMT was enabled at JVM startup, compare native categories over time.

```
jcmd <pid> VM.native_memory baseline  
jcmd <pid> VM.native_memory summary.diff
```

Reserved memory is address space; committed memory is made available for use. Neither necessarily equals current RSS. NMT's JVM categories help locate Metaspace, threads, code, GC, and compiler growth, but third-party native allocations may remain outside its accounting.

Exception and Safepoint Evidence

High exception rate can consume CPU and allocation even when exceptions are caught. JFR can record exception statistics and selected throw events. Distinguish expected control-flow exceptions from unexpected failures and avoid enabling very high-volume detailed events indefinitely without measuring overhead.

Safepoints coordinate VM operations that require threads to reach a known state. A pause observed by the application may include time reaching a safepoint plus work at the safepoint. Modern unified logging can expose safepoint timing.

```
-Xlog:safepoint
```

Do not label every stop as "GC pause." Class redefinition, deoptimization, thread dumps, and other VM operations can involve safepoint coordination.

Compiler and Code-Cache Diagnosis

JIT compilation improves hot code but creates warm-up and deoptimization behavior. During startup, latency and CPU may differ from steady state. JFR compilation and code-cache events can show compiler activity.

A full code cache can limit further compilation and reduce performance. Confirm code-cache usage before changing its size. Excessive dynamic class generation or uncommon traps may indicate framework behavior or unstable call-site types.

Microbenchmarks should use JMH rather than hand-written loops because warm-up, dead-code elimination, constant folding, and inlining can invalidate naive measurements.

Coordinated Omission and Load Tests

A client that waits for one response before sending the next reduces offered load during a stall and can under-report latency. This is coordinated omission.

Production-like tests should model arrival rate, concurrency, payload distribution, downstream latency, and failure. Record client-side latency and server-side evidence together. A profiler result from an unrealistic benchmark can optimize the wrong bottleneck.

Observability Overhead and Safety

Every diagnostic has cost. Sampling frequency, stack depth, enabled JFR events, heap traversal, dump size, and remote access affect overhead and risk.

low continuous cost: metrics, selected logs, default JFR configuration
moderate targeted cost: short profile recording, multiple thread dumps
potentially high cost: heap dump, detailed histograms, aggressive instrumentation

The actual impact depends on workload and configuration. Rehearse diagnostic commands in a staging environment and document a production runbook with access control, retention, and cleanup.

A Symptom-to-Evidence Map

high CPU	-> CPU profile + repeated thread dumps + traffic rate
growing heap floor	-> GC logs + heap dump + dominator/root analysis
growing RSS only	-> NMT + direct buffers + OS/native tools
slow API	-> trace + JFR wall time + queues + downstream metrics
thread exhaustion	-> thread creation stacks + pool metrics + -Xss + limits
lock contention	-> JFR lock events + owner stacks
frequent GC	-> allocation profile + live set + heap sizing
startup slowdown	-> class loading + JIT/compiler + dependency initialization

Use the map to choose evidence, not to jump directly to a fix.

Production Investigation Workflow

1. State the user-visible symptom and affected scope. 2. Align timestamps across metrics, logs, traces, GC, and JFR. 3. Form a falsifiable hypothesis such as "requests wait for the JDBC pool." 4. Collect the least intrusive evidence that tests it. 5. Find the limiting resource and ownership path. 6. Mitigate with awareness of side effects and preserve evidence. 7. Fix the capacity, lifecycle, algorithm, or dependency policy. 8. Reproduce and compare the same measurements after the change.

Avoid changing heap size, pool size, GC flags, and timeouts simultaneously. Multiple changes destroy causal understanding and may move failure elsewhere.

Sampling, Instrumentation, and Measurement Bias

Sampling periodically observes stacks and estimates where time is spent. Instrumentation adds probes to selected events or method boundaries. Sampling usually has lower overhead and scales better for broad profiling; instrumentation can count exact occurrences for a narrow question but may distort hot paths.

sampling: inspect every interval -> statistical picture
instrumentation: record every selected event -> detailed count with probe cost

A sample profile can miss extremely short rare events, while instrumentation can change scheduling and contention enough to hide the original behavior. Safepoint-biased sampling can overrepresent locations where threads can be observed safely; modern profilers and JFR execution sampling reduce particular biases in different ways. Always record profiler mode, event source, frequency, stack depth, and duration so the result can be reproduced.

Percentages are conditional on the captured samples. If serialization is 50 percent of a profile collected during an incident, it means half of those samples landed in that stack; it does not prove serialization consumes half of all wall-clock request latency. Relate samples to total CPU utilization, core count, traffic, and the time window.

CPU Saturation, Throttling, and Run-Queue Delay

A Java process can be slow when its measured CPU usage is below the apparent machine capacity. In containers, CPU quota throttling can suspend runnable threads after the quota is consumed. On a busy host, runnable threads can wait in the operating-system run queue.

```
application has runnable work
|
+--> executing on CPU      profiler collects samples
+--> waiting for CPU/quota  latency grows without Java blocking stack
```

Compare process CPU, container throttled time, host saturation, runnable thread count, and request latency. Adding executor threads to a CPU-throttled service increases scheduling and queueing without adding CPU. Concurrent GC and JIT compiler threads also compete within the quota, potentially lengthening both collection cycles and application work.

Wall-Clock Profiles Reveal Waiting Ownership

CPU profiles are ideal for consumed processor time. Wall-clock or elapsed-time profiles can sample threads while running or waiting and reveal latency ownership across locks, sockets, parks, and queues. The result can be dominated by legitimately idle threads, so filter by relevant thread pools, request context, or state.

```
slow request elapsed time
20 ms CPU parsing
15 ms CPU business logic
800 ms waiting for connection pool
140 ms database response
```

A CPU-only profile sees mainly the first two entries and can lead engineers to optimize 35 milliseconds while ignoring 940 milliseconds of waiting. Combine elapsed evidence with traces, pool metrics, and dependency telemetry. Conversely, a trace shows span duration but may not reveal which local method burned CPU; that is where profiles add value.

Continuous JFR as a Flight Recorder

The operational value of JFR is strongest when a bounded recording is already running. Disk-backed recordings can retain a rolling time window through maximum age and size settings. On alert, dump the current repository without stopping the recording.

```
jcmd <pid> JFR.start name=continuous settings=default \
    disk=true maxage=30m maxsize=512m

jcmd <pid> JFR.dump name=continuous \
    filename=/diagnostics/incident-%p-%t.jfr
```

The exact command options should be confirmed with `jcmd <pid> help JFR.start` on the target JDK. The default configuration is intended for low-overhead continuous use; a short profile recording can be added when deeper samples are required. Dumping a running recording does not inherently stop it.

Retention, disk permissions, cleanup, time synchronization, and artifact upload must be designed before the incident. A perfect recording that is overwritten, inaccessible in an ephemeral container, or timestamped against a different clock is not useful.

Custom JFR Events Connect Domain and Runtime

Applications can emit custom JFR events for expensive domain operations without filling ordinary logs. A carefully designed event might record operation name, batch size, tenant category, retry count, and outcome while avoiding sensitive payloads and unbounded strings.

```
@Name("com.example.BatchCompleted")
@Label("Batch Completed")
class BatchEvent extends Event {
    int itemCount;
    int retryCount;
    boolean successful;
}
```

Custom events let an investigator correlate a business operation with allocation, GC, lock, and I/O activity in one timeline. Event thresholds and stack traces control cost. Do not emit one rich event for every tiny request without measuring volume, and do not store secrets or high-cardinality payloads.

Percentiles, Histograms, and Cardinality

An average hides tail behavior. A service can have a healthy 20 ms mean while a small but important population experiences five-second latency. Use histograms capable of representing the required range and precision, and examine p50, p95, p99, and maximum with sample counts.

Percentiles cannot normally be averaged across nodes or time windows. Aggregate the underlying histogram distribution when the metrics system supports it. Also distinguish request latency from queue wait, dependency time, execution time, and retries; one end-to-end number does not identify ownership.

Labels improve segmentation but unbounded cardinality can destabilize the monitoring system. Do not use raw user IDs, request IDs, URLs with path parameters, exception messages, or query text as metric labels. Put high-cardinality identifiers in traces or structured logs with controlled retention, and keep metric dimensions bounded.

Build an Incident Evidence Bundle

For repeatable diagnosis, preserve a small manifest with every artifact:

```
UTC start/end and node identity
JDK vendor/build and effective JVM command line
container CPU/memory limits and throttling
deployment version and configuration revision
traffic and affected endpoint summary
JFR/GC log/thread dump/heap or NMT artifact names
commands used and expected impact
```

This prevents later confusion about which process, JDK, or interval a recording represents. Capture checksums and restrict access for artifacts containing application data. Use a stable clock source so traces, JFR, GC logs, and infrastructure metrics can be aligned.

From Correlation to Causation

Two events occurring together do not prove one caused the other. GC frequency and latency may both rise because traffic doubled. Lock contention may be downstream of a slow database call under the lock. High allocation may result from retries triggered by dependency failure.

Form a mechanism-based hypothesis: "database latency lengthens the lock hold, causing request threads to block and the executor queue to grow." Seek evidence for every edge, mitigate the proposed cause, and verify that predicted metrics change. Healthy-node comparison, controlled load, feature isolation, and

one-variable experiments strengthen causal confidence.

A Strong Interview Explanation

Begin with the question each evidence type answers. Metrics show trends and scope, traces show request paths, CPU and wall-clock profiles show sampled resource ownership, thread dumps show instantaneous stacks and locks, heap dumps show retention, GC logs show collector behavior, and NMT categorizes HotSpot native memory. JFR is valuable because it correlates many JVM events over one time window, while JMC helps analyze that recording.

Then explain the method: establish the incident interval and healthy baseline, state a falsifiable hypothesis, choose the least intrusive diagnostic, account for sampling bias and tool overhead, and correlate by timestamp. Distinguish CPU consumption from run-queue or quota delay and creation from retention. Preserve an evidence manifest, change one significant variable, and verify the same service-level measurements. That is production profiling as an engineering discipline rather than a collection of commands.

Interview-Ready Summary

JVM diagnosis combines metrics, logs, traces, profiles, and dumps. `jcmd` exposes target-JDK diagnostic commands. JFR records time-correlated JVM and application events, and JMC analyzes the recording. CPU profiles identify sampled execution, allocation profiles identify creation sites, heap dumps identify retention, thread dumps show current stacks and locks, and NMT categorizes HotSpot native memory.

Choose tools from the hypothesis and understand their overhead. Capture multiple thread dumps, correlate GC with application latency, distinguish heap from native memory, and compare against a healthy baseline. Evidence-driven tuning changes one meaningful variable and verifies the same service-level outcome.

Revision Notes

- Metrics identify when and where a symptom occurs.
- Profiles show sampled resource consumption over time.
- Dumps provide detailed point-in-time state.
- Establish incident time window and healthy baseline first.
- Use `jcmd <pid> help` for target-JDK command availability.
- Diagnostic commands have different operational impact.
- JFR is a built-in event-recording framework.
- Default and profile JFR configurations trade detail and overhead.
- Continuous bounded recording preserves pre-incident evidence.
- JMC findings are hypotheses, not automatic root causes.
- CPU time and wall-clock time answer different questions.
- Flame-graph width represents sample frequency.
- Allocation profiles show creation; heap dumps show retention.
- Capture several thread dumps to identify persistent stacks.
- WAITING pool threads may simply be idle.
- Java 21 ThreadMXBean does not fully monitor virtual threads.
- Correlate GC logs and JFR by timestamp.
- Heap histograms do not provide full retaining paths.
- Heap dumps are sensitive and potentially intrusive.
- NMT must be enabled at startup and is not complete native accounting.
- Not every safepoint-related pause is garbage collection.
- JIT warm-up and code-cache behavior affect performance.
- Use JMH for reliable Java microbenchmarks.
- Account for coordinated omission in load testing.
- Change one important variable and verify the same baseline metrics.
- Sampling estimates hot stacks; instrumentation records selected events with probe cost.
- Profile percentages are conditional on the captured sample population.
- CPU quota and run-queue delay can slow runnable Java threads.
- Wall-clock profiles expose waiting that CPU profiles intentionally omit.
- Continuous bounded JFR preserves evidence from before an alert.
- A JFR dump can be taken while the recording continues.
- Custom JFR events can correlate domain work with runtime behavior.
- Percentiles should be aggregated from distributions, not averaged blindly.
- Keep metric-label cardinality bounded.
- Preserve JDK, limits, commands, and timestamps with diagnostic artifacts.
- Correlation becomes causal confidence only through a supported mechanism and verification.